

Trabajo Práctico Final

Jalil Maria Emilia, Antonic Suarez Ana

1. Teórico

Consideremos un vector aleatorio $(\mathbf{x}, Y)$, donde $\mathbf{x} \in X \subset \mathbb{R}^p$ representa el vector de covariables y Y la clase, que toma valores en $\{0, 1\}$. Un clasificador es una función $g : X \rightarrow Y$. Cuando recibimos un nuevo punto $\mathbf{x}$, la predicción correspondiente es $g(\mathbf{x})$.

(a) Consideramos el Error de Clasificación Medio:

$$L(g) = \mathbb{P}(g(\mathbf{x}) \neq Y).$$

Queremos demostrar que, debido a que Y es binaria, este error coincide con el Error Cuadrático Medio:

$$L(g) = \mathbb{E}[(Y - g(\mathbf{x}))^2].$$

Demostración:

Dado que tanto Y como $g(\mathbf{x})$ sólo pueden tomar valores en $\{0, 1\}$, analicemos el valor de $(Y - g(\mathbf{x}))^2$. Las posibilidades son:

- Si $g(\mathbf{x}) = Y$, entonces $Y - g(\mathbf{x}) = 0$, y su cuadrado también.
- Si $g(\mathbf{x}) \neq Y$, entonces $Y - g(\mathbf{x}) = \pm 1$, y su cuadrado es 1.

Por lo tanto:

$$(Y - g(\mathbf{x}))^2 = \begin{cases} 0, & g(\mathbf{x}) = Y, \\ 1, & g(\mathbf{x}) \neq Y. \end{cases}$$

En otras palabras, $(Y - g(\mathbf{x}))^2 = \mathbb{1}_{\{g(\mathbf{x}) \neq Y\}}$, donde $\mathbb{1}_{\{\cdot\}}$ denota la función indicadora. Esto muestra que el cuadrado de la diferencia actúa exactamente como indicadora del evento de clasificación incorrecta.

Al tomar la esperanza de esta variable discreta se obtiene:

$$\mathbb{E}[(Y - g(\mathbf{x}))^2] = 0 \cdot \mathbb{P}(g(\mathbf{x}) = Y) + 1 \cdot \mathbb{P}(g(\mathbf{x}) \neq Y).$$

El primer término desaparece, y queda:

$$\mathbb{E}[(Y - g(\mathbf{x}))^2] = \mathbb{P}(g(\mathbf{x}) \neq Y) = L(g).$$

Con esto se prueba la igualdad pedida.

(b) Supongamos que las distribuciones condicionales son normales multivariadas, es decir,

$$\mathbf{x} | Y = 1 \sim \mathcal{N}(\mu_1, \Sigma_1), \quad \mathbf{x} | Y = 0 \sim \mathcal{N}(\mu_0, \Sigma_0).$$

Probar que la regla óptima resulta

$$g^{op}(\mathbf{x}) = \begin{cases} 1 & \text{si } r_1(\mathbf{x}) \leq r_0(\mathbf{x}) + 2 \log \frac{\pi_1}{\pi_0} + \log \frac{|\Sigma_0|}{|\Sigma_1|}, \\ 0 & \text{en c.c.} \end{cases}$$

donde, para $i = 0, 1$, se tiene que $\pi_i = P(Y = i)$, $r_i(\mathbf{x}) = (\mathbf{x} - \mu_i)^T \Sigma_i^{-1} (\mathbf{x} - \mu_i)$, y $|\Sigma_i|$ denota el determinante de la matriz Σ_i .

Demostración:

Por el teorema de Bayes, la regla óptima que minimiza el error de clasificación consiste en clasificar según la clase de mayor probabilidad posterior. Es decir, la regla óptima de Bayes escoge la clase con mayor probabilidad posterior:

$$g^{op}(\mathbf{x}) = \arg \max_{i=0,1} \mathbb{P}(Y = i | \mathbf{x}) = \arg \max_{i=0,1} f_{\mathbf{x}|Y=i}(\mathbf{x}) \pi_i.$$

La densidad normal multivariada es:

$$f_i(\mathbf{x}) = \frac{1}{(2\pi)^{p/2} |\Sigma_i|^{1/2}} \exp \left(-\frac{1}{2} r_i(\mathbf{x}) \right).$$

El término $(2\pi)^{-p/2}$ aparece en ambas densidades, por lo que se cancela cuando las comparamos.

Aplicamos logaritmo a ambos lados (válido ya que el logaritmo es estrictamente creciente):

$$\log(f_1(\mathbf{x})\pi_1) \geq \log(f_0(\mathbf{x})\pi_0).$$

Reemplazamos:

$$-\frac{1}{2}r_1(\mathbf{x}) - \frac{1}{2}\log|\Sigma_1| + \log\pi_1 \geq -\frac{1}{2}r_0(\mathbf{x}) - \frac{1}{2}\log|\Sigma_0| + \log\pi_0.$$

Pasamos todo al lado izquierdo:

$$-\frac{1}{2}r_1(\mathbf{x}) + \frac{1}{2}r_0(\mathbf{x}) - \frac{1}{2}\log|\Sigma_1| + \frac{1}{2}\log|\Sigma_0| + (\log\pi_1 - \log\pi_0) \geq 0.$$

Multiplicamos por -2 :

$$r_1(\mathbf{x}) - r_0(\mathbf{x}) - (\log|\Sigma_0| - \log|\Sigma_1|) - 2(\log\pi_1 - \log\pi_0) \leq 0.$$

Usando $\log(a) - \log(b) = \log(a/b)$ y pasando todos los términos excepto $r_1(\mathbf{x})$ al lado derecho:

$$r_1(\mathbf{x}) \leq r_0(\mathbf{x}) + 2 \log \frac{\pi_1}{\pi_0} + \log \frac{|\Sigma_0|}{|\Sigma_1|}.$$

Y llegamos exactamente a la regla óptima pedida:

$$g^{op}(\mathbf{x}) = \begin{cases} 1 & \text{si } r_1(\mathbf{x}) \leq r_0(\mathbf{x}) + 2 \log \frac{\pi_1}{\pi_0} + \log \frac{|\Sigma_0|}{|\Sigma_1|}, \\ 0 & \text{en c.c.} \end{cases}$$

(c) Probar que si $\Sigma_0 = \Sigma_1 = \Sigma$, entonces la regla del ítem anterior resulta

$$g^{op}(\mathbf{x}) = \begin{cases} 1 & \text{si } D_1(\mathbf{x}) - 2 \log(\pi_1) \leq D_0(\mathbf{x}) - 2 \log(\pi_0), \\ 0 & \text{en c.c.} \end{cases}$$

donde $D_i(\mathbf{x}) = (\mathbf{x} - \mu_i)^T \Sigma^{-1} (\mathbf{x} - \mu_i)$.

Demostración:

Partimos de la regla general obtenida en (b):

$$r_1(\mathbf{x}) \leq r_0(\mathbf{x}) + 2 \log \frac{\pi_1}{\pi_0} + \log \frac{|\Sigma_0|}{|\Sigma_1|}.$$

Si $\Sigma_0 = \Sigma_1 = \Sigma$, entonces:

$$\frac{|\Sigma_0|}{|\Sigma_1|} = 1 \quad \Rightarrow \quad \log(1) = 0.$$

Por lo tanto:

$$r_1(\mathbf{x}) \leq r_0(\mathbf{x}) + 2 \log \frac{\pi_1}{\pi_0}.$$

Además, como ambas covarianzas son iguales, se cumple que

$$r_i(\mathbf{x}) = (\mathbf{x} - \mu_i)^T \Sigma^{-1} (\mathbf{x} - \mu_i) = D_i(\mathbf{x}).$$

Reemplazando:

$$D_1(\mathbf{x}) \leq D_0(\mathbf{x}) + 2 \log \frac{\pi_1}{\pi_0}.$$

Usamos la identidad:

$$2 \log \left(\frac{\pi_1}{\pi_0} \right) = 2(\log \pi_1 - \log \pi_0),$$

y reordenamos:

$$D_1(\mathbf{x}) - 2 \log(\pi_1) \leq D_0(\mathbf{x}) - 2 \log(\pi_0).$$

Interpretación : al ser $\Sigma_0 = \Sigma_1$, los términos cuadráticos en $\mathbf{x}$ que aparecen al expandir $D_1(\mathbf{x})$ y $D_0(\mathbf{x})$ se cancelan mutuamente, resultando en una frontera de decisión lineal (un hiperplano). Este resultado corresponde al modelo de LDA.

(d) Comprobar que si $\pi_0 = \pi_1$, entonces la regla del ítem anterior resulta

$$g^{op}(\mathbf{x}) = \begin{cases} 1 & \text{si } D_1(\mathbf{x}) \leq D_0(\mathbf{x}) \\ 0 & \text{en c. c.} \end{cases}$$

es decir: se clasifica a $\mathbf{x}$ en la población cuya media está más cerca en distancia de Mahalanobis. Si $\Sigma = \sigma^2 I_p$ (siendo I_p la identidad de $p \times p$), ¿a qué equivaldría esta regla?

Demostración:

Partimos de la regla obtenida en (c):

$$D_1(\mathbf{x}) \leq D_0(\mathbf{x}) + 2 \log \frac{\pi_1}{\pi_0}.$$

Si $\pi_1 = \pi_0$, entonces

$$\frac{\pi_1}{\pi_0} = 1 \quad \Rightarrow \quad \log(1) = 0,$$

y la condición se reduce a:

$$D_1(\mathbf{x}) \leq D_0(\mathbf{x}).$$

Esto significa que clasificamos $\mathbf{x}$ según la media más cercana en distancia de Mahalanobis.

Caso donde $\Sigma = \sigma^2 I_p$

Si la matriz de covarianzas es de la forma:

$$\Sigma = \sigma^2 I_p \quad \Rightarrow \quad \Sigma^{-1} = \frac{1}{\sigma^2} I_p,$$

lo cual corresponde a variables no correlacionadas con igual varianza σ^2 , entonces:

$$D_i(\mathbf{x}) = (\mathbf{x} - \mu_i)^T \Sigma^{-1} (\mathbf{x} - \mu_i) = \frac{1}{\sigma^2} \|\mathbf{x} - \mu_i\|^2.$$

Como el factor $\frac{1}{\sigma^2}$ es común a ambas expresiones, se cancela en la comparación:

$$D_1(\mathbf{x}) \leq D_0(\mathbf{x}) \quad \Leftrightarrow \quad \|\mathbf{x} - \mu_1\|^2 \leq \|\mathbf{x} - \mu_0\|^2.$$

Interpretación: Cuando $\pi_0 = \pi_1$, la regla de clasificación depende únicamente de cuál media está más cerca en distancia de Mahalanobis. Si además $\Sigma = \sigma^2 I_p$, la distancia de Mahalanobis es proporcional a la euclídea.

(e) Mostrar que si Σ_0 y Σ_1 no coinciden, entonces la regla óptima puede escribirse como

$$g^{op}(\mathbf{x}) = \begin{cases} 1 & \text{si } \mathbf{x}^t A \mathbf{x} + b^t \mathbf{x} + c \leq 0 \\ 0 & \text{en c. c.} \end{cases}$$

y hallar las expresiones de A , b y c .

Demostración:

Partimos de la condición obtenida en (b):

$$r_1(\mathbf{x}) - r_0(\mathbf{x}) \leq 2 \log \frac{\pi_1}{\pi_0} + \log \frac{|\Sigma_0|}{|\Sigma_1|}.$$

Definimos el término constante:

$$K = 2 \log \frac{\pi_1}{\pi_0} + \log \frac{|\Sigma_0|}{|\Sigma_1|}.$$

Expandimos cada distancia de Mahalanobis usando la identidad:

$$(\mathbf{x} - \mu)^T M (\mathbf{x} - \mu) = \mathbf{x}^T M \mathbf{x} - 2\mu^T M \mathbf{x} + \mu^T M \mu.$$

Entonces:

$$r_1(\mathbf{x}) - r_0(\mathbf{x}) = \mathbf{x}^T (\Sigma_1^{-1} - \Sigma_0^{-1}) \mathbf{x} - 2(\Sigma_1^{-1} \mu_1 - \Sigma_0^{-1} \mu_0)^T \mathbf{x} + (\mu_1^T \Sigma_1^{-1} \mu_1 - \mu_0^T \Sigma_0^{-1} \mu_0).$$

Imponiendo la condición $r_1(\mathbf{x}) - r_0(\mathbf{x}) \leq K$, identificamos:

$$A = \Sigma_1^{-1} - \Sigma_0^{-1},$$

$$b = -2(\Sigma_1^{-1} \mu_1 - \Sigma_0^{-1} \mu_0),$$

$$c = \mu_1^T \Sigma_1^{-1} \mu_1 - \mu_0^T \Sigma_0^{-1} \mu_0 - K.$$

donde $A \in \mathbb{R}^{p \times p}$ es una matriz simétrica, $b \in \mathbb{R}^p$ es un vector, y $c \in \mathbb{R}$ es un escalar.

Observación : si $\Sigma_0 = \Sigma_1$, entonces $A = 0$ y recuperamos la regla lineal del ítem (c).

Interpretación: como las matrices de covarianza no coinciden, aparece un término cuadrático $\mathbf{x}^T A \mathbf{x}$ que no se cancela. Por lo tanto, la frontera resultante es curvada (una cuádrica), característica del clasificador QDA.

2. Práctico

```
# Setear el directorio correspondiente
# setwd("~/Proyectos facultad/R Visualización/Tp vis")

# Librerías utilizadas:
knitr::opts_chunk$set(echo = TRUE)
library(ggplot2)
library(plotly)
```

```
##
## Adjuntando el paquete: 'plotly'

## The following object is masked from 'package:ggplot2':
##
##     last_plot

## The following object is masked from 'package:stats':
##
##     filter

## The following object is masked from 'package:graphics':
##
##     layout

library(patchwork)
library(MASS)
```

```
##
## Adjuntando el paquete: 'MASS'

## The following object is masked from 'package:patchwork':
##
##     area

## The following object is masked from 'package:plotly':
##
##     select
```

```
library(dplyr)
```

```
##
## Adjuntando el paquete: 'dplyr'

## The following object is masked from 'package:MASS':
##
##     select

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(tidyr)
set.seed(1234)
```

Parte A

En esta actividad vamos a trabajar con los datos disponibles en data `wdbc_X.csv`, que contiene 569 observaciones que corresponden a variables relacionadas con imágenes de lesiones de pacientes con diferentes pronósticos de cáncer de mama.

```
datos <- read.csv("data_wdbc_X.csv", header = TRUE, sep = ";")
etiquetas <- read.csv("data_wdbc_y.csv", header = TRUE, sep = ";")

datos_completos <- merge(datos, etiquetas, by = "id")
```

Consigna 1

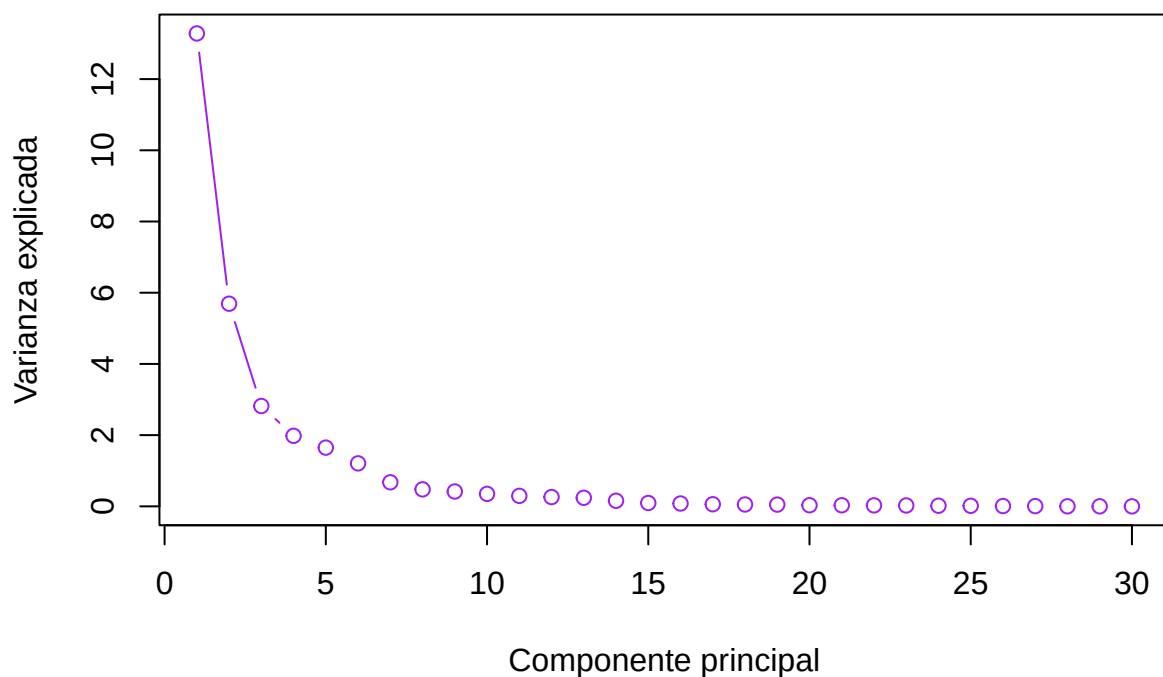
Hacer un análisis de componentes principales a partir de los datos estandarizados. Analizar la proporción de varianza explicada por las primeras componentes. Hacer un scree-plot. Luego, graficar las observaciones: en el espacio de las primeras dos componentes principales, y en el espacio de las primeras tres.

```
# Estandarizamos y centramos los datos
datos_std <- scale(datos_completos[, -c(1, ncol(datos_completos))])

PCA <- prcomp(datos_std)
var_exp <- PCA$sdev^2

plot(var_exp, type = "b", xlab = "Componente principal", ylab = "Varianza explicada", main = "Scree Plot")
```

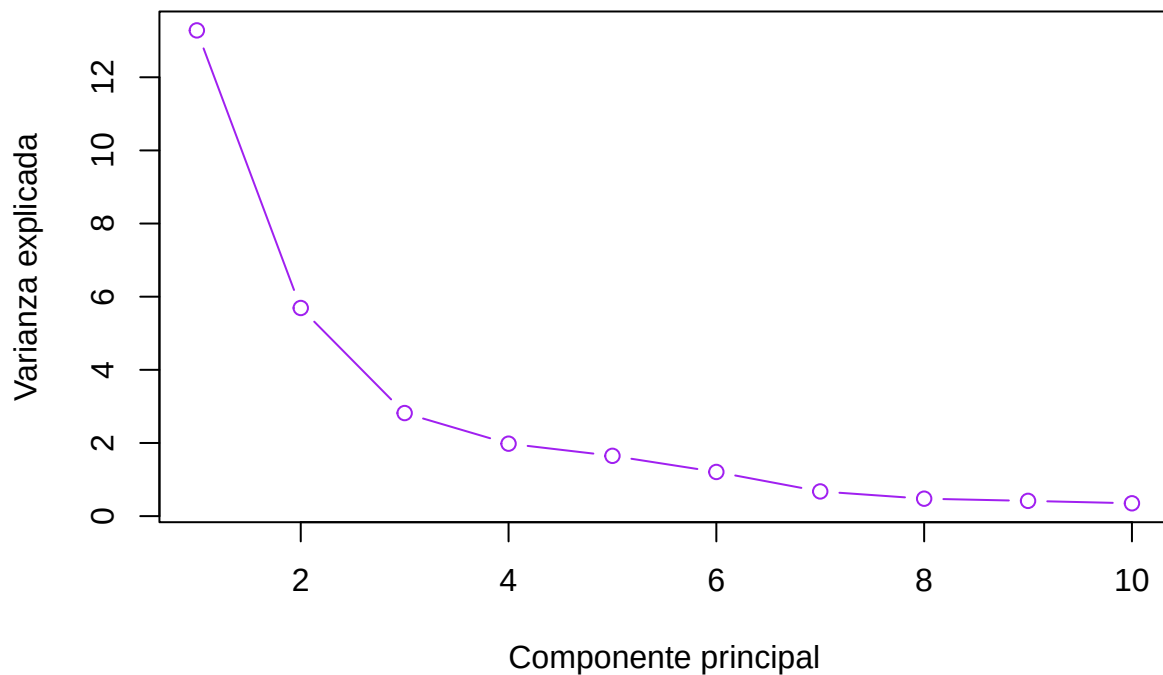
Scree Plot del PCA con todas las componentes principales



Hagamos foco en las primeras 10 componentes:

```
plot(var_exp[1:10], type = "b", xlab = "Componente principal", ylab = "Varianza explicada", main = "Scree Plot del PCA con las primeras 10 componentes principales")
```

Scree Plot del PCA con las primeras 10 componentes principales



Parecería que las componentes que más varianza aportan son las primeras 4-5 aproximadamente.

```
var_exp_1 <- var_exp[1] / sum(var_exp)
var_exp_2 <- var_exp[2] / sum(var_exp)
var_exp_3 <- var_exp[3] / sum(var_exp)
var_exp_4 <- var_exp[4] / sum(var_exp)
var_exp_5 <- var_exp[5] / sum(var_exp)

tabla_var <- data.frame(
  Componente = paste0("PC", 1:5),
  Var_Explicada = round(c(var_exp_1, var_exp_2, var_exp_3, var_exp_4, var_exp_5), 4)*100,
  Var_Acumulada = round(c(var_exp_1, var_exp_1 + var_exp_2, var_exp_1 + var_exp_2 + var_exp_3, var_exp_1 + var_exp_2 + var_exp_3 + var_exp_4, var_exp_1 + var_exp_2 + var_exp_3 + var_exp_4 + var_exp_5), 4)*100)

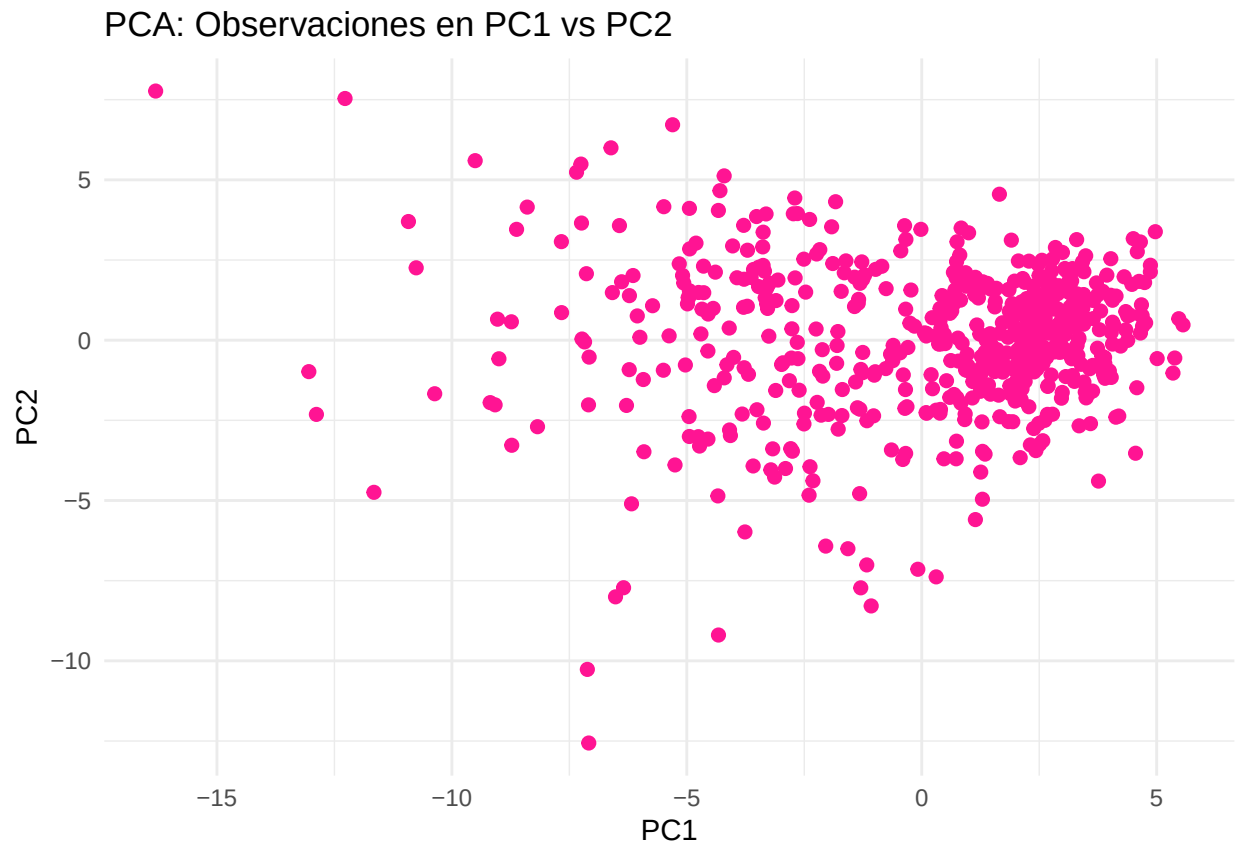
tabla_var
```

##	Componente	Var_Explicada	Var_Acumulada
## 1	PC1	44.27	44.27
## 2	PC2	18.97	63.24
## 3	PC3	9.39	72.64
## 4	PC4	6.60	79.24
## 5	PC5	5.50	84.73

Observamos que con solo 4 componentes principales ya retenemos cerca del 80% de la variabilidad total de los datos (y con 5 componentes casi el 85%). En este caso, PCA resulta especialmente útil, ya que permite resumir la mayor parte de la variabilidad de las 30 variables originales en solo unas pocas componentes.

```
scores <- PCA$x

# Graficamos los datos sobre las primeras 2
ggplot(scores, aes(x = PC1, y = PC2)) +
  geom_point(size = 2, color = "deeppink") +
  theme_minimal() +
  labs(title = "PCA: Observaciones en PC1 vs PC2")
```



```
# Graficamos los datos sobre las primeras 3
plot_ly(x = scores[,1], y = scores[,2], z = scores[,3],
  type = "scatter3d", mode = "markers",
  marker = list(size = 4, color = "deeppink")) %>%
  layout(title = "PCA: Observaciones en PC1 vs PC2 vs PC3")
```

Consigna 2

Aplicar un clustering por k-means con $k = 2$ clusters usando:

- Las primeras dos componentes
- Las primeras tres componentes

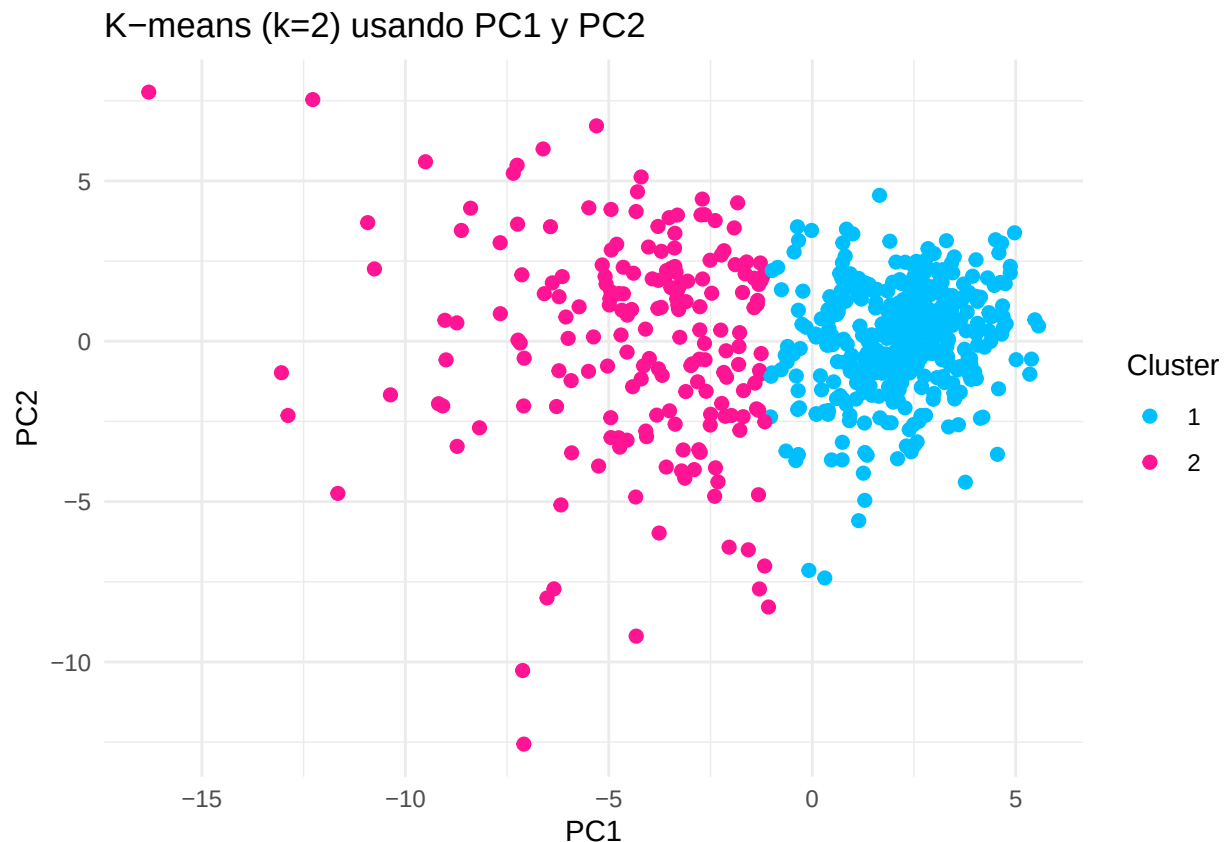
Visualizar los resultados del clustering en:

- El gráfico de las primeras dos componentes coloreando los puntos según el cluster asignado por k-means del inciso (a).
- El gráfico de las primeras tres componentes coloreando los puntos según el cluster asignado por k-means del inciso (b).

Evaluar si $k = 2$ es una cantidad razonable de grupos en todos los casos.

```
kmeans_2d <- kmeans(scores[, 1:2], centers = 2, nstart = 100)

ggplot(scores, aes(x = PC1, y = PC2, color = factor(kmeans_2d$cluster))) +
  geom_point(size = 2) +
  theme_minimal() +
  scale_color_manual(values = c("deepskyblue", "deeppink")) +
  labs(title = "K-means (k=2) usando PC1 y PC2",
       color = "Cluster")
```



```
kmeans_3d <- kmeans(scores[, 1:3], centers = 2, nstart = 100)

plot_ly(x = scores[,1], y = scores[,2], z = scores[,3], type = "scatter3d", mode = "markers",
        color = factor(kmeans_3d$cluster), colors = c("deepskyblue", "deeppink"), marker = list(size = 4)) %>
```

Al visualizar los datos en la primeras 2 y 3 componentes principales no se observan dos grupos claramente separados. Aunque k-means (con $k=2$) genera una partición de los datos en dos grupos, la separación no es

visualmente evidente. Esto sugiere que la estructura natural de los datos no se ajusta a dos clusters bien diferenciados. Aún así, con un poco más de detalle, puede identificarse cierta tendencia: uno de los grupos forma una nube más compacta y densa, mientras que el otro aparece algo más disperso y alejado.

Parte B

Para lo que sigue, vamos a trabajar con los datos disponibles en data `wdbc_y.csv` que contiene el diagnóstico de cada una de las observaciones del dataset data `wdbc_x.csv`. Este diagnóstico es “B” o “M”, según si el tumor fue diagnosticado como benigno o maligno, respectivamente.

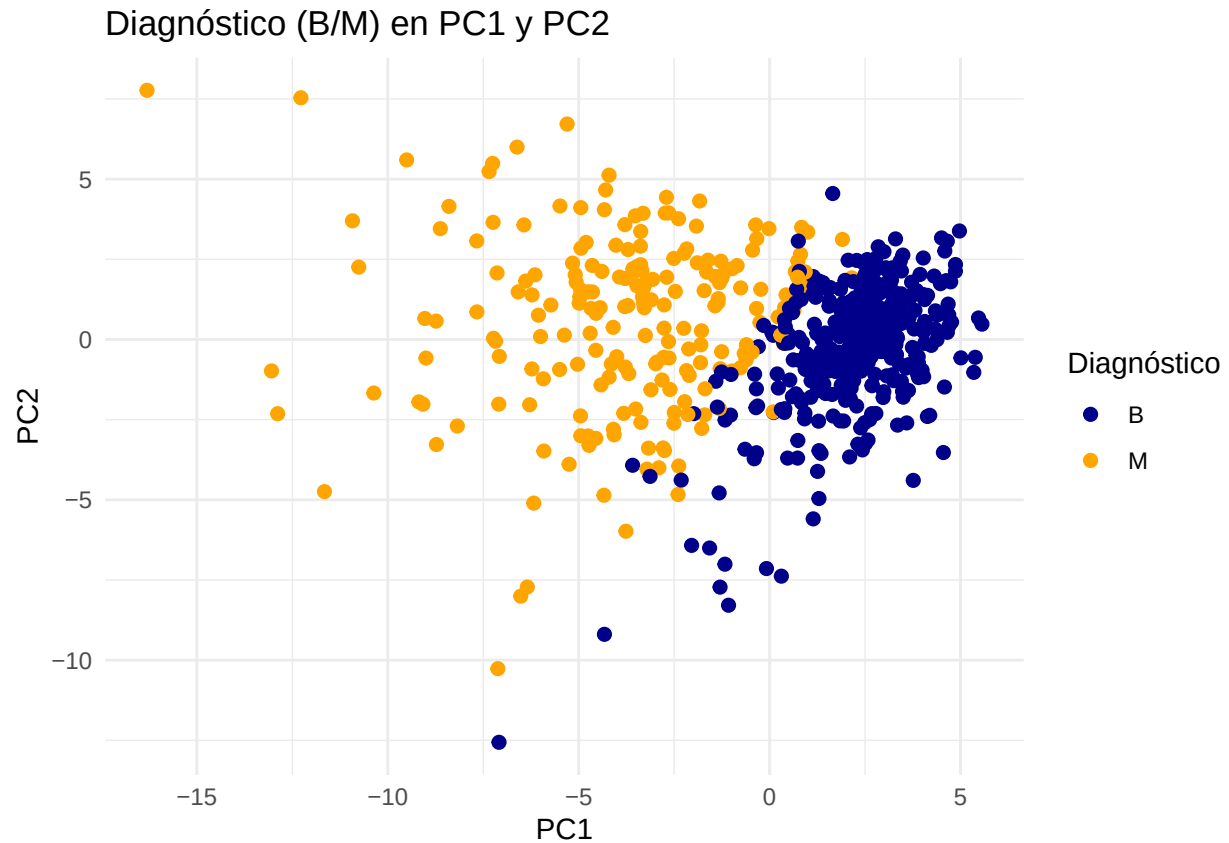
Consigna 3

Para el clustering hecho sobre las dos primeras componentes:

- Representar en gráficos separados las observaciones coloreadas según diagnóstico (B/M) y cluster (1/2). En particular, observar qué tipo de frontera parece separar a los datos según diagnóstico.
- Construir una tabla de contingencia entre diagnóstico (B/M) y cluster (1/2).
- Suponiendo que cada cluster representa la clase mayoritaria dentro de él, calcular el porcentaje de observaciones correctamente “clasificadas” por el clustering . ¿Qué tan bien el “clustering ciego” separa benignos de malignos?
- Repetir para el clustering hecho sobre las tres primeras componentes y comparar los resultados.

```
diagnostico <- datos_completos$diagnosis

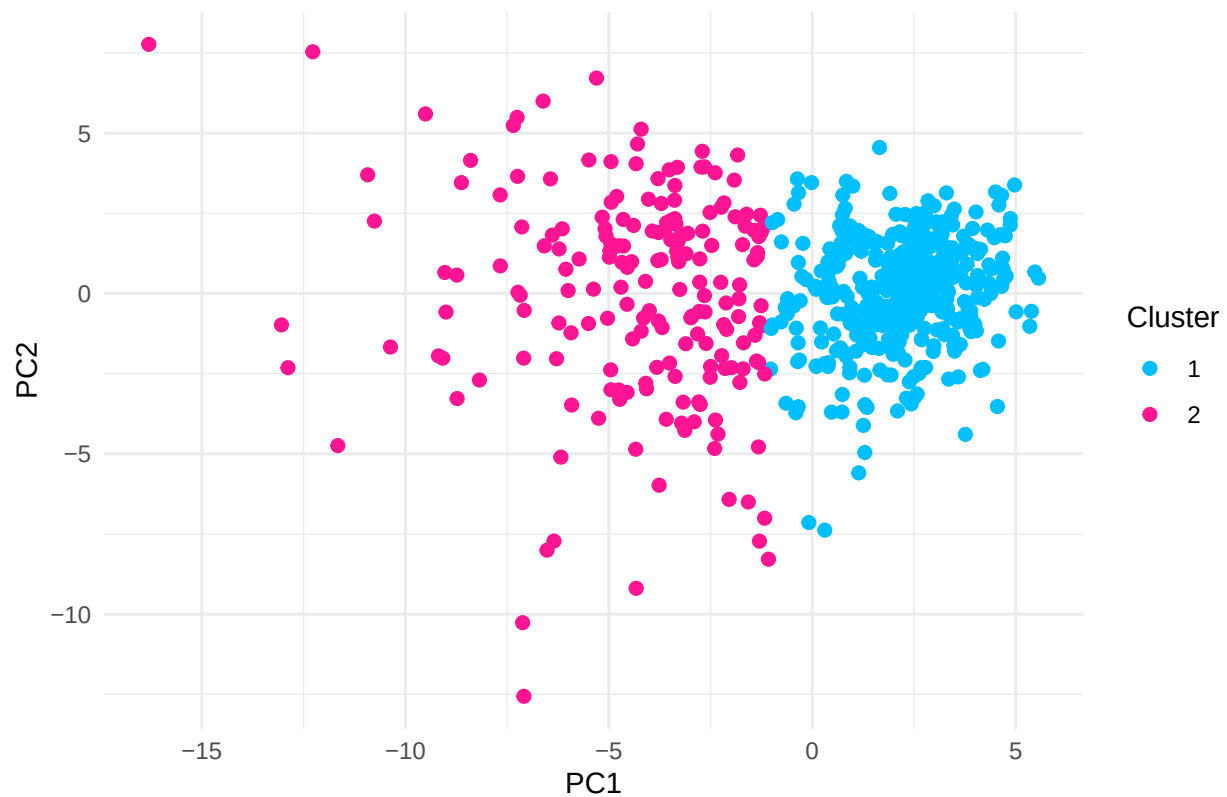
ggplot(scores, aes(PC1, PC2, color = diagnostico)) +
  geom_point(size = 2) +
  scale_color_manual(values = c("B" = "darkblue", "M" = "orange")) +
  theme_minimal() +
  labs(title = "Diagnóstico (B/M) en PC1 y PC2", color = "Diagnóstico")
```



Darí la impresión de que la frontera que separa a los datos es lineal, pero las clases no son completamente separables, ya que hay superposición entre ambas.

```
ggplot(scores, aes(PC1, PC2, color = factor(kmeans_2d$cluster))) +  
  geom_point(size = 2) +  
  scale_color_manual(values = c("1" = "deepskyblue", "2" = "deeppink")) +  
  theme_minimal() +  
  labs(title = "Clusters k-means (k=2) en PC1 y PC2", color = "Cluster")
```

Clusters k-means (k=2) en PC1 y PC2



```
tabla_contingencia_2d <- table(diagnostico, kmeans_2d$cluster)
tabla_contingencia_2d
```

```
##
## diagnostico    1    2
##               B 341  16
##               M  37 175
```

Entonces, al clasificar según la clase mayoritaria (con la semilla actual), las observaciones que caen en el cluster 1 serían clasificadas como “B” y las que caen en el cluster 2 como “M”.

```
acc_2d <- sum(diag(tabla_contingencia_2d)) / sum(tabla_contingencia_2d) * 100
cat("Accuracy:", round(acc_2d, 2), "%\n")
```

```
## Accuracy: 90.69 %
```

El “clustering ciego” separa con un accuracy mayor al 90%.

Pasemos a las 3 componentes principales:

```
plot_ly(
  x = scores[,1], y = scores[,2], z = scores[,3],
  type = "scatter3d", mode = "markers",
  color = diagnostico,
```

```

colors = c("B" = "darkblue", "M" = "orange"),
marker = list(size = 3)
) %>% layout(title = "Diagnóstico (B/M) en PC1, PC2 y PC3")

```

Nuevamente la frontera parece ser un hiperplano. Las clases siguen sin ser separables, incluso en una dimensión mayor.

```

plot_ly(
  x = scores[,1], y = scores[,2], z = scores[,3],
  type = "scatter3d", mode = "markers",
  color = factor(kmeans_3d$cluster),
  colors = c("1" = "deepskyblue", "2" = "deeppink"),
  marker = list(size = 3)
) %>% layout(title = "Clusters k-means (k=2) en PC1, PC2 y PC3")

```

Como pasaba con las primera 2 componentes, los gráficos se ven bastante similares, es decir, parecerían separar la mayoría de las observaciones en las mismas regiones, excepto en algunos casos.

```

tabla_contingencia_3d <- table(diagnostico, kmeans_3d$cluster)
tabla_contingencia_3d

```

```

##
## diagnostico    1    2
##               B 343  14
##               M  37 175

```

Una vez más, si tuviéramos que asignarle a cada cluster la clase mayoritaria (con la semilla actual), clasificaríamos a las observaciones en el cluster 1 como “B” y a las del cluster 2 como “M”. Si comparamos con la tabla de contingencia anterior, podemos ver que únicamente 2 observaciones más pasarían a estar correctamente clasificadas al agregar la 3er componente principal para el k-means.

```

acc_3d <- sum(diag(tabla_contingencia_3d)) / sum(tabla_contingencia_3d) * 100
cat("Accuracy:", round(acc_3d, 2), "%\n")

```

```
## Accuracy: 91.04 %
```

```
cat("Accuracy mejoró en un", round(acc_3d - acc_2d, 2), "%")
```

```
## Accuracy mejoró en un 0.35 %
```

Consigna 4

En lo que sigue, se busca construir modelos supervisados que sí usen el diagnóstico. Para ello, fijar una semilla y dividir los datos en 80%-20 % para los grupos de entrenamiento y testeo.

- Hacer PCA sobre el conjunto de entrenamiento y clasificar por LDA usando solo las primeras 2 componentes principales. Calcular el accuracy en el conjunto de testeo. Importante: al calcular los scores en el conjunto de testeo, tener en cuenta que deben usarse las componentes principales calculadas a partir del conjunto de entrenamiento. ¿Por qué? Explicar brevemente.

- Repetir lo anterior usando:
 - solo las primeras 5 componentes principales
 - solo las primeras 10 componentes principales
- Comparar los tres modelos (2, 5 y 10 componentes) en términos de precisión y complejidad.
- Variar el número de componentes principales entre las primeras 2 a las primeras 30.

Graficar la precisión del modelo en función de las componentes. ¿Cuántas componentes elegiría para un modelo final que balancee simplicidad y rendimiento? (Puede ser útil evaluar cuán sensible es este análisis según la partición train/test que se tenga. Se pueden probar otras semillas y ver cómo cambia este gráfico.)

```
# Fijamos otra semilla para este ejercicio
set.seed(2025)

# Separamos los datos
n <- nrow(datos_completos)
idx_train <- sample(1:n, size = 0.8 * n)

train <- datos_completos[idx_train, ]
test  <- datos_completos[-idx_train, ]

# Separamos los atributos de las etiquetas
X_train <- train[, -c(1, ncol(train))]
X_test  <- test[,  -c(1, ncol(test))]

y_train <- train[, c(ncol(train))]
y_test  <- test[,  c(ncol(test))]
```

Para escalar los datos antes de aplicar PCA, debemos calcular y guardar la media y la desviación estándar solo del conjunto de entrenamiento, y usar esos valores para estandarizar también el conjunto de test. Esto evita introducir información del test durante el entrenamiento. Por la misma razón, el PCA debe ajustarse únicamente con los datos de entrenamiento (sus componentes principales se obtienen del train y luego el test se proyecta usando esas mismas componentes). Esto asegura que no haya **data leakage** y que la evaluación en el conjunto de test sea válida.

```
train_mean <- apply(X_train, 2, mean)
train_sd   <- apply(X_train, 2, sd)

X_train_std <- scale(X_train, center = train_mean, scale = train_sd)
X_test_std  <- scale(X_test,  center = train_mean, scale = train_sd)
```

Hacemos PCA sobre el conjunto de train y proyectamos el conjunto de test sobre las mismas.

```
PCA_train <- prcomp(X_train_std)

scores_train <- PCA_train$x
loadings_train <- PCA_train$rotation

scores_test <- X_test_std %*% loadings_train
```

LDA con las primeras 2 componentes:

```
lda_2comp <- lda(diagnosis ~ ., data = data.frame(scores_train[, 1:2], diagnosis = y_train))

pred_2 <- predict(lda_2comp, newdata = data.frame(scores_test[, 1:2]))
acc_2 <- mean(pred_2$class == y_test) * 100

cat("Accuracy con 2 Componentes principales:", round(acc_2, 4), "%\n")
```

Accuracy con 2 Componentes principales: 93.8596 %

Con las primeras 5:

```
lda_5comp <- lda(diagnosis ~ ., data = data.frame(scores_train[, 1:5], diagnosis = y_train))

pred_5 <- predict(lda_5comp, newdata = data.frame(scores_test[, 1:5]))
acc_5 <- mean(pred_5$class == y_test) * 100

cat("Accuracy con 5 Componentes principales:", round(acc_5, 4), "%\n")
```

Accuracy con 5 Componentes principales: 93.8596 %

Con las primeras 10:

```
lda_10comp <- lda(diagnosis ~ ., data = data.frame(scores_train[, 1:10], diagnosis = y_train))

pred_10 <- predict(lda_10comp, newdata = data.frame(scores_test[, 1:10]))
acc_10 <- mean(pred_10$class == y_test) * 100

cat("Accuracy con 10 Componentes principales:", round(acc_10, 4), "%\n")
```

Accuracy con 10 Componentes principales: 93.8596 %

Cuando comparamos los tres modelos, vemos que usar 5 o 10 componentes principales no mejora nada la precisión respecto de usar solo 2. Esto indica que la mayor parte de la información útil para separar casos benignos de malignos ya está contenida en las primeras dos componentes, y que agregar más solo suma variación que no ayuda a clasificar mejor.

Como los modelos con más componentes también son más complejos, y no ofrecen una mejora real, lo más razonable es quedarse con el modelo más simple: el que usa solo 2 componentes principales. Es igual de preciso, más fácil de visualizar y menos propenso al sobreajuste (Navaja de Ockham).

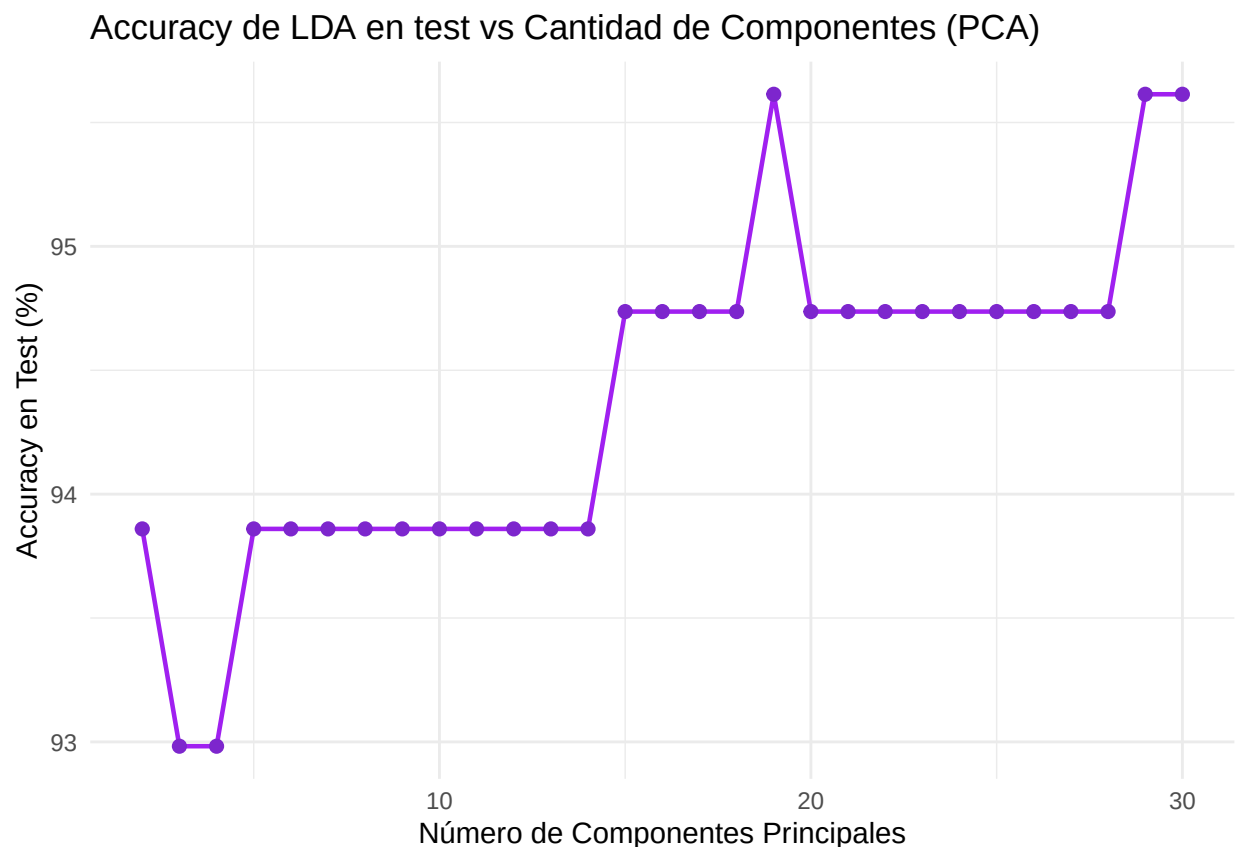
```
accuracies <- numeric(29)
components <- 2:30

for (k in components) {
  # Repetimos lo que veníamos haciendo pero de forma genérica
  train_k <- data.frame(scores_train[, 1:k], diagnosis = y_train)
  lda_k <- lda(diagnosis ~ ., data = train_k)
  test_k <- data.frame(scores_test[, 1:k])
  pred_k <- predict(lda_k, newdata = test_k)
  accuracies[k - 1] <- mean(pred_k$class == y_test) * 100
}
```



```
df_acc <- data.frame(
  Componentes = components,
  Accuracy = accuracies
)

ggplot(df_acc, aes(x = Componentes, y = Accuracy)) +
  geom_line(linewidth = 0.8, color = "purple") +
  geom_point(size = 2, color = "purple3") +
  theme_minimal() +
  labs(
    title = "Accuracy de LDA en test vs Cantidad de Componentes (PCA)",
    x = "Número de Componentes Principales",
    y = "Accuracy en Test (%)"
  )
)
```



En el gráfico se ve que la precisión no mejora de forma pareja cuando agregamos más componentes. De hecho, al pasar de 2 a 3 componentes la performance incluso baja un poco, lo que sugiere que algunas componentes nuevas no ayudan y solo agregan ruido. También aparecen algunos picos (por ejemplo cuando usamos 19 componentes) donde la precisión sube. Eso probablemente pasa porque esas componentes sí contienen algo de información útil para distinguir benignos de malignos. En resumen, no todas las componentes aportan lo mismo: algunas ayudan, otras no, y por eso el gráfico tiene esa forma irregular.

Para no sesgar nuestras conclusiones a una semilla específica, le pedimos a la IA que nos genere un pipeline basado en lo que hicimos anteriormente pero permitiéndonos probar varias semillas y graficar todo junto.

```
#####
# PCA + LDA vs cantidad de componentes (2 a 30)
# Varias semillas - gráfico comparativo
#####

#-----
# 0. Preparamos X e y desde tus datos
#-----

X <- datos_completos[, -c(1, ncol(datos_completos))] # todas menos ID y diagnosis
y <- datos_completos$diagnosis # diagnóstico

#-----
# 1. Función que ejecuta todo el pipeline para una semilla
#-----

run_pca_lda <- function(seed, X, y, min_comp = 2, max_comp = 30) {
  set.seed(seed)

  # Split 80/20
  train_idx <- sample(seq_len(nrow(X)), size = 0.8 * nrow(X))
  X_train <- X[train_idx, ]
  X_test <- X[-train_idx, ]
  y_train <- y[train_idx]
  y_test <- y[-train_idx]

  # Escalado basado SOLO en train
  means <- apply(X_train, 2, mean)
  sds <- apply(X_train, 2, sd)

  X_train_scaled <- scale(X_train, center = means, scale = sds)
  X_test_scaled <- scale(X_test, center = means, scale = sds)

  # PCA solo con train
  pca <- prcomp(X_train_scaled)

  # vector accuracies (solo 2 a 30)
  accuracies <- numeric(max_comp)

  for (k in min_comp:max_comp) {

    # Scores del train
    train_scores <- pca$x[, 1:k]

    # Proyección del test en las mismas componentes
    test_scores <- X_test_scaled %*% pca$rotation[, 1:k]

    # Armamos datasets
    df_train <- data.frame(diagnosis = y_train, train_scores)
    df_test <- data.frame(diagnosis = y_test, test_scores)

    # LDA
    model_lda <- lda(diagnosis ~ ., data = df_train)
  }
}
```

```

    pred <- predict(model_lda, df_test)

    accuracies[k] <- mean(pred$class == y_test)
  }

  return(accuracies)
}

#-----
# 2. Ejecutamos para varias semillas
#-----

seeds <- c(1, 1234, 12345, 2025)

results_list <- lapply(seeds, function(s) run_pca_lda(s, X, y, min_comp = 2, max_comp = 30))

#-----
# 3. Convertimos a formato tidy para graficar
#-----

df_results <- do.call(cbind, results_list)
colnames(df_results) <- paste0("seed_", seeds)

df_plot <- data.frame(components = 1:30, df_results) %>%
  pivot_longer(-components, names_to = "seed", values_to = "accuracy") %>%
  filter(components >= 2) # <-- acá filtramos 2 a 30

#-----
# 4. Gráfico
#-----

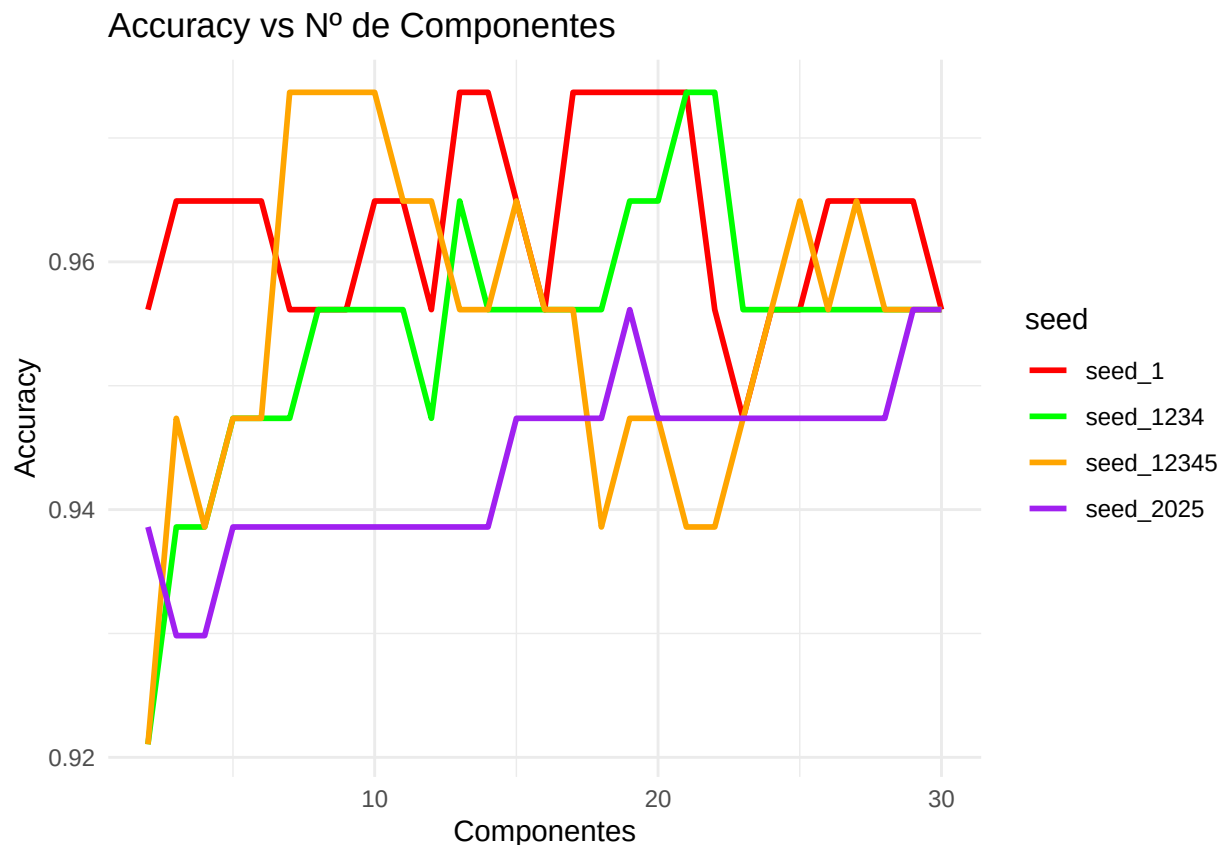
ggplot(df_plot, aes(x = components, y = accuracy, color = seed)) +
  geom_line(size = 1) +
  scale_color_manual(values = c(
    "seed_1" = "red",
    "seed_1234" = "green",
    "seed_12345" = "orange",
    "seed_2025" = "purple"
  )) +
  labs(
    title = "Accuracy vs N° de Componentes",
    x = "Componentes",
    y = "Accuracy"
  ) +
  theme_minimal()

```

```

## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

```



En este gráfico podemos ver, primero, que los resultados dependen bastante de la partición inicial entre entrenamiento y test: distintas semillas generan curvas con formas diferentes. Esto muestra que el rendimiento del modelo no es completamente estable frente a cambios en la separación de los datos.

También vemos que, a diferencia de lo que sugerían las primeras pruebas con una sola partición, en varios casos aumentar de 2 a 5 o 10 componentes sí mejora la precisión de manera notable. Es decir, para algunas divisiones de los datos, usar más componentes realmente aporta información útil que ayuda a separar benignos de malignos.

Por otro lado, a pesar de esas variaciones y algunos picos, la precisión se mantiene en general dentro de un rango relativamente alto (aproximadamente entre 92 % y 98 %), lo cual indica que el modelo basado en PCA + LDA es bastante robusto en promedio.

En resumen, el gráfico muestra que el número óptimo de componentes no es fijo y puede depender de la división de los datos, aunque el rendimiento es consistentemente bueno. Si el objetivo es un modelo estable y sencillo, elegir pocas componentes (por ejemplo entre 5 y 10) puede ser un buen compromiso entre simplicidad y precisión.